

Fundos no Umbrel

via Portainer — sem terminal, tudo pela interface web

O que você vai aprender

- O que é Portainer e por que ele simplifica o gerenciamento de containers
- Instalar Portainer dentro do Umbrel App Store
- Primeiro acesso e criação do usuário admin
- Subir o Fundos como Stack (3 caminhos diferentes)
- Gerenciar variáveis de ambiente, logs e atualizações
- Acessar pelo iPhone via Tailscale
- Backup e recuperação · troubleshooting

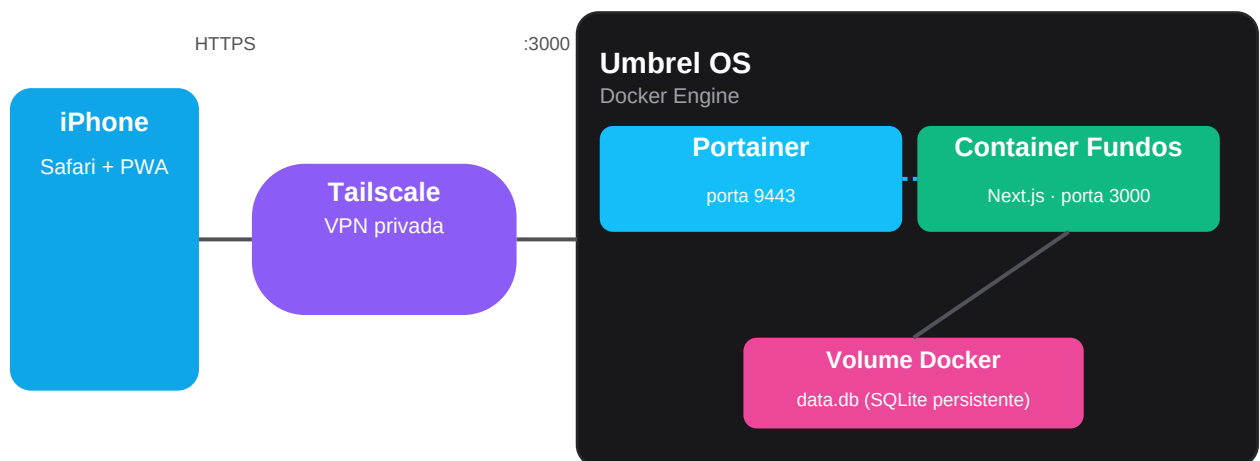
Versão 1.0 · Tempo estimado: 25 minutos

1. O que é o Portainer

O **Portainer** é uma **interface web para gerenciar Docker** sem precisar do terminal. Você cria, atualiza, vê logs, faz backup e monitora containers tudo pelo navegador. Funciona em Linux, Windows, Mac, Raspberry Pi, NAS — qualquer lugar que rode Docker.

Por que usar Portainer no Umbrel? O Umbrel já tem App Store, mas para apps **customizados** (como o Fundos) a UI nativa é limitada. O Portainer entra como uma camada de gerenciamento mais poderosa: você pode subir qualquer *docker-compose.yml* com 3 cliques, ver logs em tempo real, e atualizar com 1 clique.

1.1 Como o Portainer se encaixa no Umbrel



Arquitetura completa: iPhone → Tailscale → Umbrel (Portainer + Container)

O Portainer fica entre **você** e o **Docker Engine do Umbrel**. Não é mais um container especial — é só uma UI bonita pro Docker que já existe.

1.2 Diferenças em relação ao deploy via SSH

Aspecto	SSH + docker compose	Portainer
Aprendizado	Precisa saber comandos Docker	GUI guiada, formulários
Deploy	Comandos no terminal	3 cliques na UI
Logs	docker compose logs	Aba 'Logs' tempo real
Atualização	git pull + rebuild	Botão 'Pull and redeploy'
Variáveis	Editar .env manualmente	Tela de env vars
Monitoramento	docker ps + grep	Dashboard visual com gráficos
Multi-usuário	Não	Sim, com permissões

Mobile	Acesso via SSH	UI responsiva no browser
--------	----------------	--------------------------

2. Instalar o Portainer no Umbrel

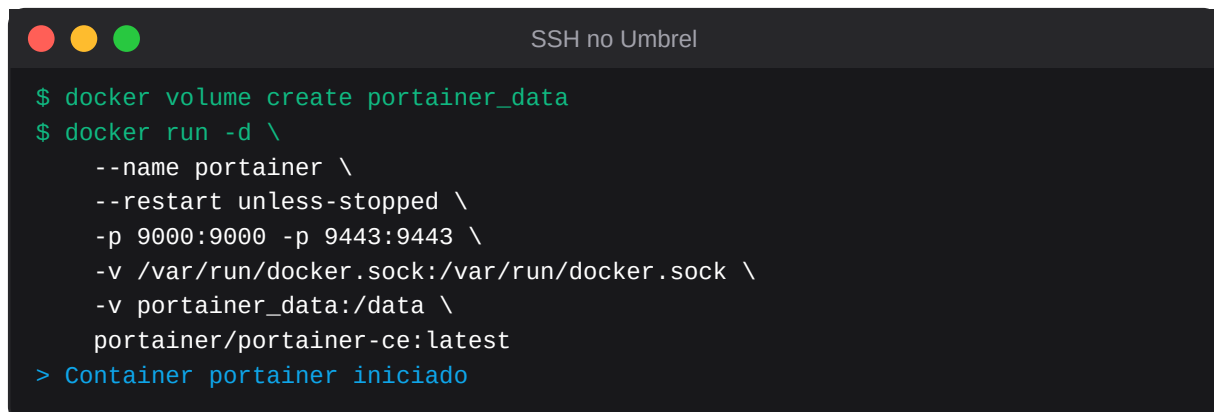
Existem 2 caminhos. Recomendo o A (Umbrel App Store) — é o mais simples.

2.1 Caminho A — App Store do Umbrel (recomendado)

1. Acesse a UI do Umbrel no navegador (geralmente `http://umbrel.local`)
2. Toque no ícone **App Store** no menu inferior
3. Use a busca: digite **Portainer**
4. Toque em **Install** no card do Portainer CE
5. Aguarde ~30 segundos. Quando aparecer **Open**, está pronto

2.2 Caminho B — Instalação manual via SSH

Se a App Store do Umbrel não tem Portainer (algumas versões antigas) ou você prefere instalação manual:

A terminal window titled "SSH no Umbrel" with a dark background and light green text. It shows the commands to create a Docker volume and run the Portainer container. The output indicates the container is started.

```
$ docker volume create portainer_data
$ docker run -d \
  --name portainer \
  --restart unless-stopped \
  -p 9000:9000 -p 9443:9443 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data \
  portainer/portainer-ce:latest
> Container portainer iniciado
```

Atenção: O volume `/var/run/docker.sock` dá ao Portainer **acesso total ao Docker** do host. Trate o Portainer como um administrador root. Use uma senha forte (próximo passo).

3. Primeiro acesso e setup do admin

Abra no seu navegador (PC, Mac ou iPhone via Tailscale):

`https://<ip-do-umbrel>;9443`

(ou `http://...:9000` se preferir sem HTTPS)

Atenção: Vai aparecer aviso de certificado **auto-assinado**. É normal — o Portainer gera um cert local. Clique em **Avançado** → **Acessar mesmo assim**. Em produção com Tailscale Serve você pode ter cert válido (veja seção 9).

3.1 Criar usuário admin

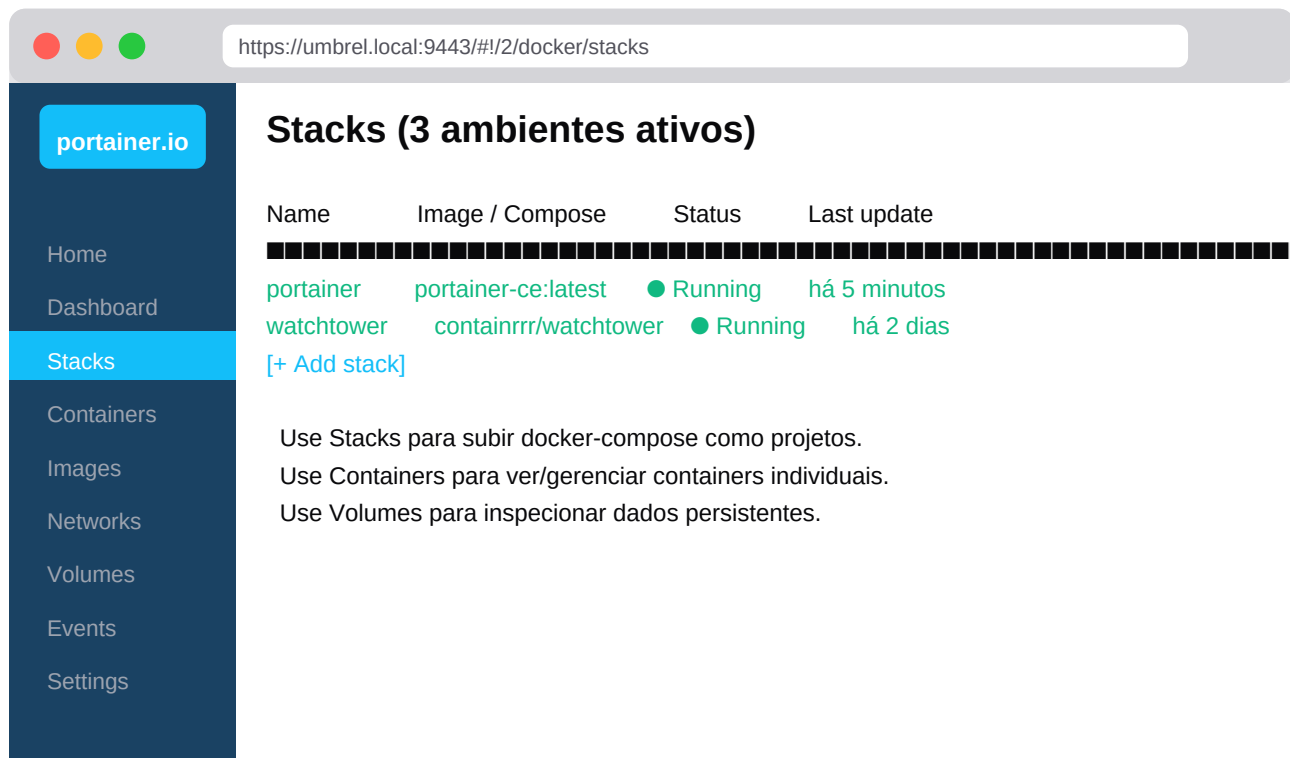
Na primeira tela, o Portainer pede pra criar o usuário admin:

1. **Username:** admin (ou o que preferir)
2. **Password:** mínimo 12 caracteres — use senha forte
3. **Confirm password:** repita
4. Clique **Create user**

3.2 Selecionar ambiente

Na tela seguinte, escolha **Get Started** (não 'Add environments'). Isso conecta com o Docker local — exatamente o que queremos para gerenciar o Umbrel.

3.3 Tour rápido da interface



Tela inicial do Portainer — menu lateral à esquerda, conteúdo principal à direita

Itens importantes do menu lateral:

Menu	Para que serve
Home / Dashboard	Visão geral, RAM/CPU, containers ativos
Stacks	Onde você vai gerenciar o Fundos (= docker compose)
Containers	Lista de todos os containers individuais
Images	Imagens Docker baixadas, espaço usado
Networks	Redes Docker, raramente precisa mexer
Volumes	Dados persistentes — onde o data.db do Fundos vive
Events	Histórico do que aconteceu no Docker
Settings	Configurações do Portainer (usuários, registries)

4. Subir o Fundos como Stack

No Portainer, **uma Stack = um docker-compose.yml**. Vamos criar uma para o Fundos. Há 3 caminhos — recomendo o **Repository** (caminho A), que é o mais robusto e fácil de atualizar depois.



Fluxo do deploy via Git Repository

4.1 Caminho A — Git Repository (recomendado)

Vantagens: atualizações com 1 clique, versionamento via git, e o Portainer pode até puxar automaticamente quando você der *git push*.

Pré-requisitos:

- Seu projeto Fundos no GitHub (público ou privado)
- Arquivo docker-compose.yml na raiz
- Dockerfile na raiz (já vem pronto)

Passos no Portainer:

1. Menu lateral → **Stacks**
2. Botão **+ Add stack** no topo
3. Preencha conforme o formulário abaixo
4. Botão **Deploy the stack** no final

Create stack

Stacks > Add stack

Name *

fundos

Build method

Web editor

Upload

Repository

Custom template

Repository URL *

https://github.com/SEU-USUARIO/fundos

Repository reference

refs/heads/main

Compose path

docker-compose.yml

Environment variables

Definidas em .env do repo OU adicionadas aqui:

Formulário 'Add stack' do Portainer com configuração para o Fundos

APP_PASSWORD

5842

Deploy the stack

Detalhes de cada campo:

Campo	Valor a digitar	Observação
Name	fundos	minúsculo, sem espaço
Build method	Repository	última coluna
Repository URL	https://github.com/SEU/fundos	troque SEU pelo seu user
Repository auth	(deixe vazio se público)	ou token Personal Access
Repository reference	refs/heads/main	ou refs/heads/master
Compose path	docker-compose.yml	padrão, geralmente OK
Auto update	(opcional)	ativa polling do git
Polling interval	5m	se Auto update on
Environment variables	APP_PASSWORD = 5842	+ Add another variable
	TZ = America/Sao_Paulo	

Dica: Repositório privado? Crie um **Personal Access Token** no GitHub (Settings → Developer settings → Personal access tokens → Tokens classic) com permissão repo. Ative a opção *Authentication* no formulário e cole o token.

4.2 Caminho B — Web editor (cola o YAML)

Mais simples se você só quer testar uma vez. Sem versionamento.

1. Stacks → + Add stack
2. Build method: **Web editor**
3. Cole o conteúdo do docker-compose.yml no editor
4. Adicione env vars: APP_PASSWORD, TZ
5. Deploy the stack

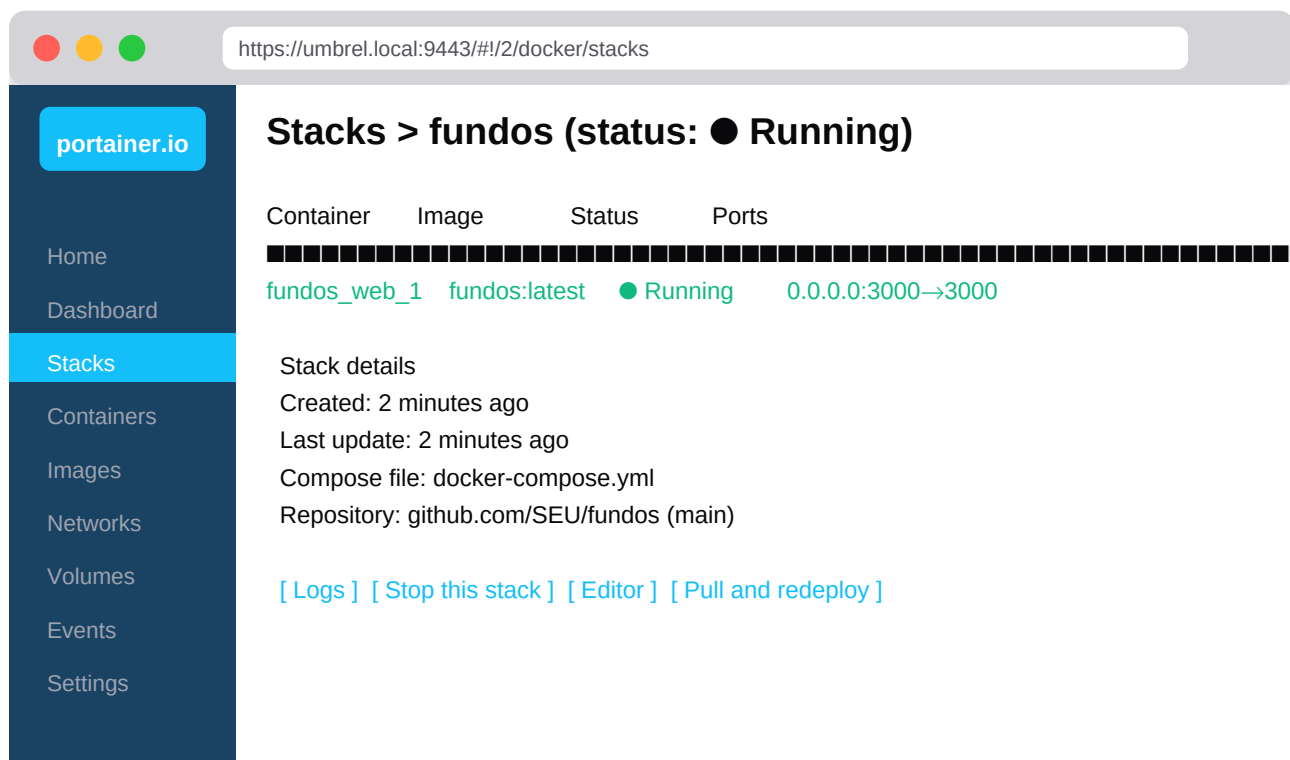
4.3 Caminho C — Upload do YAML

Igual ao B, mas você faz upload de um arquivo .yml em vez de colar.

5. O que acontece após o Deploy

Quando você clica **Deploy the stack**, o Portainer faz, em ordem:

1. Clona seu repositório git em uma pasta interna do Portainer
2. Lê o docker-compose.yml
3. Constrói a imagem Docker conforme o Dockerfile (*primeira vez demora 2-5 minutos*)
4. Cria o volume fundos_data para o banco SQLite
5. Sobe o container com as env vars que você definiu
6. Mostra o status na lista de Stacks



Stack 'fundos' rodando — você pode parar, ver logs ou redeploy a qualquer momento

5.1 Conferir que está funcionando

Em outra aba, acesse:

```
http://&lt;ip-do-umbrel&gt;;3000
```

Deve aparecer a tela de login do Fundos. Use sua senha (5842 ou a que você definiu na variável APP_PASSWORD).

5.2 Ver os logs em tempo real

Se algo der errado, ou só pra ver as requisições chegando, clique em **Containers** → **fundos_web_1** → **Logs**. Ou direto no Stack: aba **Logs**.

```
# Auto-refresh ativado
$ next-server starting...
> ▲ Next.js 15.5.18
> ✓ Ready in 3.2s
{"t":"2026-05-16T22:00:00Z","nivel":"info","msg":"scheduler.start"}
> GET / 200 in 124ms
> POST /api/lancamentos 201 in 89ms
```

6. Atualizar quando o código mudar

Esse é o maior ganho de usar Portainer: **atualização com 1 clique**.

Fluxo:

1. Você faz git push das mudanças no seu PC
2. No Portainer: Stacks → fundos → botão **Pull and redeploy**
3. Confirme. Portainer baixa o código novo, faz build, troca o container
4. O volume com o data.db é preservado. Zero perda de dados.

6.1 Auto-update (Portainer puxando sozinho)

Se ativou **Auto update** no formulário do Stack, o Portainer faz polling do git e atualiza automaticamente. Intervalo recomendado: 5-15 minutos.

Atenção: Auto-update + git push em produção = qualquer commit no main vai direto pro servidor sem teste. Considere usar uma branch prod separada e fazer pull request consciente.

6.2 Rollback se algo quebrar

Se a versão nova quebrar, faça *git revert* do commit ruim no seu repo, e no Portainer clique **Pull and redeploy** de novo. O Portainer volta pro estado anterior em segundos.

Dica: Antes de mudanças **grandes** (schema novo, nova versão major), exporta o JSON pelo app (Configurações → Exportar JSON). É seu plano B se algo der errado.

7. Gerenciar variáveis de ambiente

Senhas, tokens de API e configurações ficam em **environment variables**. No Portainer, você gerencia tudo pela UI — sem editar arquivo `.env`.

7.1 Adicionar / alterar uma variável

1. Stacks → fundos → aba **Editor** ou role até **Environment variables**
2. Clique + **Add an environment variable**
3. Digite **name** (ex: APP_PASSWORD) e **value** (ex: minha-senha)
4. Clique **Update the stack**. Portainer recria o container com a nova var.

7.2 Variáveis úteis no Fundos

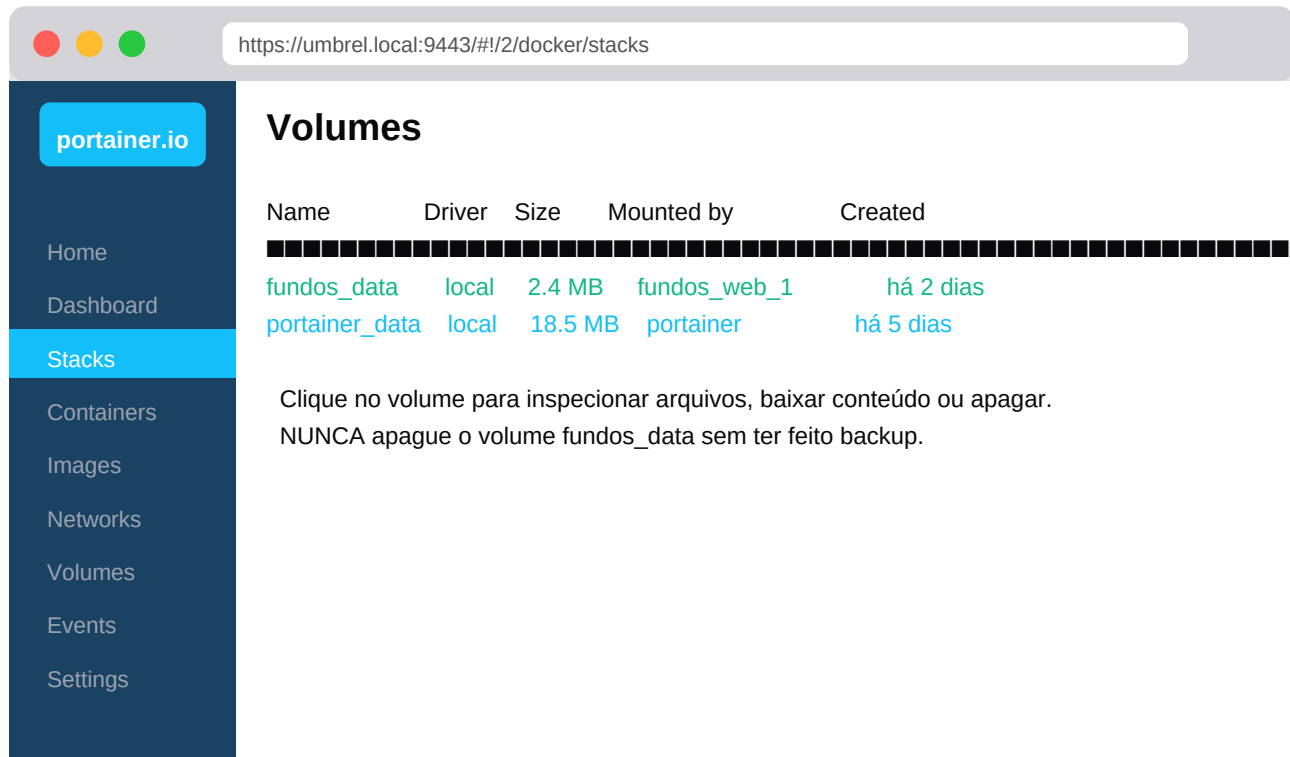
Variável	Para que serve	Exemplo
APP_PASSWORD	Senha de acesso ao app	5842
TZ	Fuso horário	America/Sao_Paulo
DATABASE_URL	Caminho do SQLite (não mude)	file:/app/data/data.db
NODE_ENV	Modo de execução (auto)	production

8. Volumes (onde mora o data.db)

O **data.db** do Fundos fica num *Docker volume* chamado fundos_data. Isso garante que ele sobreviva a updates, rebuilds e reinícios.

8.1 Ver os volumes

Menu lateral → **Volumes**. Você verá fundos_data e seu tamanho.



8.2 Baixar o conteúdo de um volume

Portainer não tem botão de download direto. Para fazer backup do volume via Portainer, use a **Console** de um container que monte esse volume:

- Containers → fundos_web_1 → ícone **Console** (terminal)
- Selecione /bin/sh e Connect
- Rode `cat /app/data/data.db | base64` para ver o conteúdo

Dica: Mas **muito mais simples**: dentro do app Fundos, vá em **Configurações** → **Exportar JSON**. Você recebe um arquivo portátil com tudo. Ou configure o **backup automático na nuvem via rclone** (veja o guia rclone).

9. Acessar pelo iPhone via Tailscale

Mesmo fluxo do guia anterior. Lembrete rápido:

1. Instale Tailscale no Umbrel (App Store ou via terminal)
2. Instale Tailscale no iPhone (App Store)
3. Faça login com a mesma conta nos dois
4. No iPhone Safari, acesse `http://<hostname-umbrel>.tailXXXX.ts.net:3000`
5. Tela de login do Fundos aparece. Logue.
6. Compartilhar → Adicionar à Tela de Início → app instalado como PWA

9.1 Acessar o Portainer do iPhone

O Portainer também é acessível via Tailscale:

```
https://&lt;hostname-umbrel&gt;.tailXXXX.ts.net:9443
```

Útil pra parar/restart containers de qualquer lugar. A UI do Portainer é **responsiva** — funciona bem no iPhone.

10. Bônus: Watchtower (updates automáticos de imagens)

Se você usa imagens pré-publicadas (em vez de build do git), o **Watchtower** monitora o registry e atualiza containers automaticamente quando aparece uma imagem nova.

Stack pronto pra colar no Portainer:

```
version: '3'
services:
  watchtower:
    image: containrrr/watchtower
    restart: unless-stopped
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    environment:
      WATCHTOWER_POLL_INTERVAL: 3600 # checa a cada hora
      WATCHTOWER_INCLUDE_RESTARTING: 'true'
      WATCHTOWER_CLEANUP: 'true' # apaga imagens antigas
    command: --label-enable
```

Marca os containers que devem ser atualizados adicionando uma label no docker-compose do Fundos:

```
services:
  web:
    image: ghcr.io/SEU/fundos:latest
    labels:
      com.centurylinklabs.watchtower.enable: 'true'
```

Daí em diante, sempre que você publicar uma imagem nova com a tag :latest, o Watchtower atualiza sozinho em até 1 hora.

Atenção: Watchtower só faz sentido se você usa imagens pré-publicadas. Se você está usando o método **Repository** do Portainer (build no servidor), use o **Auto update** do próprio Portainer.

11. Solução de problemas

Portainer não abre (timeout no 9443)

Confira: (1) o container portainer está rodando: `docker ps | grep portainer`. (2) a porta 9443 está exposta (no install via SSH, deveria estar). (3) firewall do Umbrel não está bloqueando. Tente 9000 (HTTP) como alternativa.

Stack falha no build com 'no such file Dockerfile'

Você esqueceu de subir o Dockerfile pro git. Confira no GitHub: o arquivo Dockerfile precisa estar na raiz do repositório. Faça git add + push e tente Pull and redeploy.

Build demora 10+ minutos

Primeira build é assim mesmo (baixa imagem Node, instala npm, instala todas as deps, faz prisma generate, next build). Builds seguintes ficam em 1-2 min graças ao cache do Docker.

Container fica restartando em loop

Vá em Containers → fundos_web_1 → Logs. Causas comuns: APP_PASSWORD não definida, porta 3000 em uso, banco corrompido. Veja a stacktrace exata nos logs.

Esqueci o usuário admin do Portainer

Pare o container Portainer e suba de novo com --reset-password: `docker run --rm -v portainer_data:/data portainer/helper-reset-password`. Ele dá uma senha temporária — anote, faça login e troque.

Quero migrar do método SSH+compose pro Portainer

Pare o stack antigo (`docker compose down`). NÃO apague o volume. No Portainer, crie uma stack apontando pro mesmo repo. O Portainer vai reutilizar o volume existente automaticamente — seus dados continuam intactos.

Como ver consumo de RAM/CPU?

Containers → click no nome → aba Stats. Mostra gráfico em tempo real. Útil pra debugar lentidão. O Fundos consome ~150 MB de RAM em idle.

Portainer dá 'docker socket not found'

Quando instalado via Umbrel App, normalmente está OK. Se manual: garanta que o volume -v /var/run/docker.sock:/var/run/docker.sock foi montado no comando docker run.

12. Checklist final

- Umbrel OS rodando e acessível
- Portainer instalado (via App Store ou docker run)
- Primeiro acesso em `https://...:9443` funcionou
- Usuário admin criado com senha forte
- Repositório Fundos no GitHub com Dockerfile e docker-compose.yml
- Stack 'fundos' criada via Repository no Portainer
- Variáveis APP_PASSWORD e TZ configuradas
- Status do stack: ● Running (verde)
- Container fundos_web_1: ● Up + porta 3000 mapeada
- Acesso local `http://:3000` abre o app
- Acesso via Tailscale do iPhone funciona
- App adicionado à Tela de Início (PWA)
- Login com senha entrou OK
- Volume fundos_data aparece em Volumes
- Logs do container mostram 'Next.js Ready' sem erros
- Testou 'Pull and redeploy' (atualização funciona)
- (opcional) Backup automático via rclone configurado

Com todos os ■ marcados: sua infra está completa. **Update do app = 1 clique. Logs e RAM = 1 clique. Restore = 1 clique.** A vida fica muito mais fácil com Portainer.

Fim do guia.